tf.Module Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/Module

> *Base neural network module class.*

Function Signatures

```
tf.Module( name=None )
```

Code Examples

```
tf.Module(
name=None
)
```

```
tf.Module(
name=None
)
```

```
class Dense(tf.Module):
def __init__(self, input_dim, output_size, name=None):
super().__init__(name=name)
self.w = tf.Variable(
tf.random.normal([input_dim, output_size]), name='w')
self.b = tf.Variable(tf.zeros([output_size]), name='b')
def __call__(self, x):
y = tf.matmul(x, self.w) + self.b
return tf.nn.relu(y)
```

```
class Dense(tf.Module):
```

```
def __init__(self, input_dim, output_size, name=None):
```

```
super().__init__(name=name)
```

```
self.w = tf.Variable(
```

```
tf.random.normal([input_dim, output_size]), name='w')
```

Documentation

Base neural network module class.

View aliases

Compat aliases for migration

See
Migration guide for

more details.

tf.compat.v1.Module

Used in the notebooks

• Introduction to modules, layers, and models

- Better performance with tf.function
- Distributed training with Core APIs and DTensor
- Logistic regression for binary classification with Core APIs
- Multilayer perceptrons for digit recognition with Core APIs
- Distributed training with DTensors
- DeepDream
- Simple audio recognition: Recognizing keywords
- Neural machine translation with attention
- Neural machine translation with a Transformer and Keras

A module is a named container for tf.Variables, other tf.Modules and functions which apply to user input. For example a dense layer in a neural network might be implemented as a tf.Module:

You can use the Dense layer as you would expect:

By subclassing tf.Module instead of object any tf.Variable or tf.Module instances assigned to object properties can be collected using the variables, trainable_variables or submodules property:

Subclasses of tf.Module can also take advantage of the _flatten method

which can be used to implement tracking of any other types.

All tf.Module classes have an associated tf.name_scope which can be used to group operations in TensorBoard and create hierarchies for variable names which can help with debugging. We suggest using the name scope when creating nested submodules/parameters or for forward methods whose graph you might want
to inspect in TensorBoard. You can enter the name scope explicitly using with self.name_scope: or you can annotate methods (apart from __init__) with @tf.Module.with_name_scope.

Attributes

Submodules are modules which are properties of this module, or found as properties of modules which are properties of this module (and so on).

Methods

with_name_scope

View source

Decorator to automatically enter the module name scope.

Using the above module would produce tf.Variables and tf.Tensors whose names included the module name: